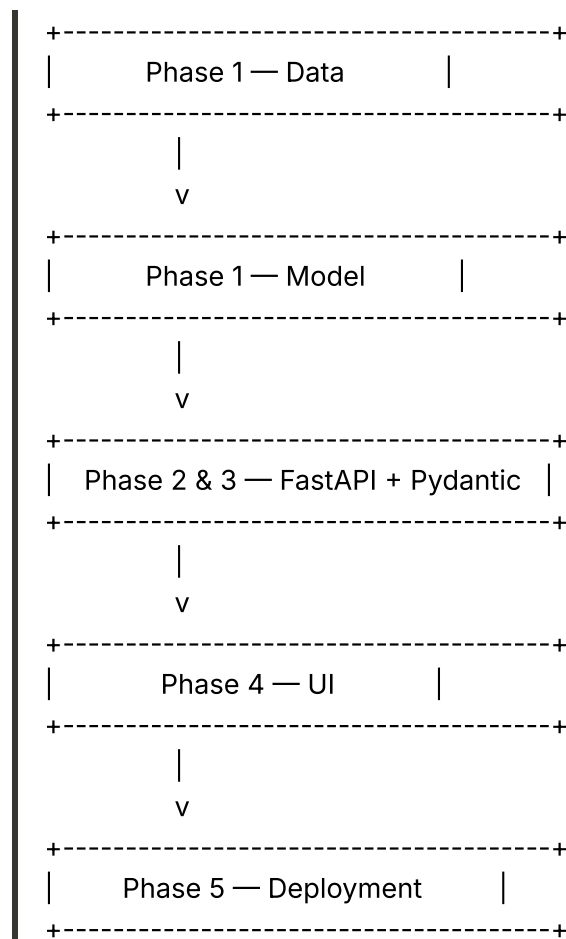




Building and Deploying a Mental Health Score Predictor

Project Execution Plan



1. Machine Learning Model Build

Everything upstream of the API.

Notebook checklist

- ☐ Import libraries - NumPy, Pandas, Matplotlib/Seaborn, Scikit-Learn, Joblib
- ☐ Load the dataset and check shape and column names
- ☐ Basic data understanding - data types, missing values, duplicate rows, summary statistics
- ☐ Exploratory Data Analysis - target distribution, correlation heatmap, stress vs score, usage vs score, sleep vs score, platform counts
- ☐ Data Cleaning - drop duplicate rows, fix an invalid negative value found in the data
- ☐ Check skewness on numeric columns, log-transform the one column that needed it
- ☐ Feature engineering - group a high-cardinality column into top categories + "Other"
- ☐ Decide encoding strategy - ordinal for ordered categories, one-hot for unordered ones
- ☐ Train/test split, done before any preprocessing to avoid data leakage
- ☐ Build preprocessing with `ColumnTransformer` - impute, scale, encode
- ☐ Wrap preprocessing + model together in a single `Pipeline`
- ☐ Train a baseline model, then a stronger model on the same pipeline
- ☐ Hyperparameter tuning with `RandomizedSearchCV`
- ☐ Evaluate and compare models with proper metrics
- ☐ Check feature importance to confirm the model learned sensible patterns
- ☐ Save the complete pipeline with `joblib.dump()`
- ☐ Run one prediction example end-to-end to sanity-check the saved pipeline



Why it matters

We save the whole pipeline, not just the raw model - the FastAPI backend can then call `predict()` directly on raw input without reimplementing any preprocessing.

2. FastAPI Service Layer

Wraps the saved pipeline behind a clean, documented HTTP interface.

Tasks checklist

- ☐ Load `model.pkl` once at startup (not per-request)
- ☐ Create a `/predict` POST route

- ☐ Mount static frontend from the same app (avoids CORS)
- ☐ Add a `/health` route for uptime checks



Important to note

Serving frontend + API from one FastAPI app means same-origin requests → no CORS headers to configure at all.

3. Pydantic Validation

The contract between frontend input and model input - reject bad data before it reaches the pipeline.

Tasks checklist

- ☐ Define a request schema with one typed field per model input
- ☐ Add validation constraints so bad values are rejected automatically
- ☐ Define a response schema for the prediction output
- ☐ Wire both schemas into the `/predict` route



Record

Send a bad request (e.g. a negative age) and show FastAPI's auto-generated 422 error - this is the moment that sells Pydantic to viewers.

4. Frontend (HTML, CSS, JS)

No framework needed - plain `fetch()` is the whole integration.

Tasks checklist

- ☐ Build a form in `index.html`
- ☐ POST the form data via `fetch` in `script.js`
- ☐ Render prediction + confidence in the DOM
- ☐ Handle loading + error states visibly

5. Deployment - Render

One web service, one repo, one build command.

Tasks checklist

- ☐ Push repo to GitHub
- ☐ New Web Service on Render, connect repo
- ☐ Build command: `pip install -r requirements.txt`
- ☐ Start command uses `$PORT`, not a hardcoded port



Record

The service spins down when idle - the first request after inactivity can take 30–50s to wake. Explain this so a live demo doesn't look broken.